PEMROGRAMAN BERORIENTASI OBJEK

STUDI KASUS BANK

Disusun oleh :

1. Haikal Rafli Sembiring (24060124130079)
2. Muhammad Zaidan Alfarizi (24060124130102)
3. Mohammad Sulthon Ariefin (24060124130104)
4. Rio Setiawan Hastanu Putra (24060124130068)

PROGRAM STUDI INFORMATIKA

FAKULTAS SAINS DAN MATEMATIKA

UNIVERSITAS DIPONEGORO

SEMARANG

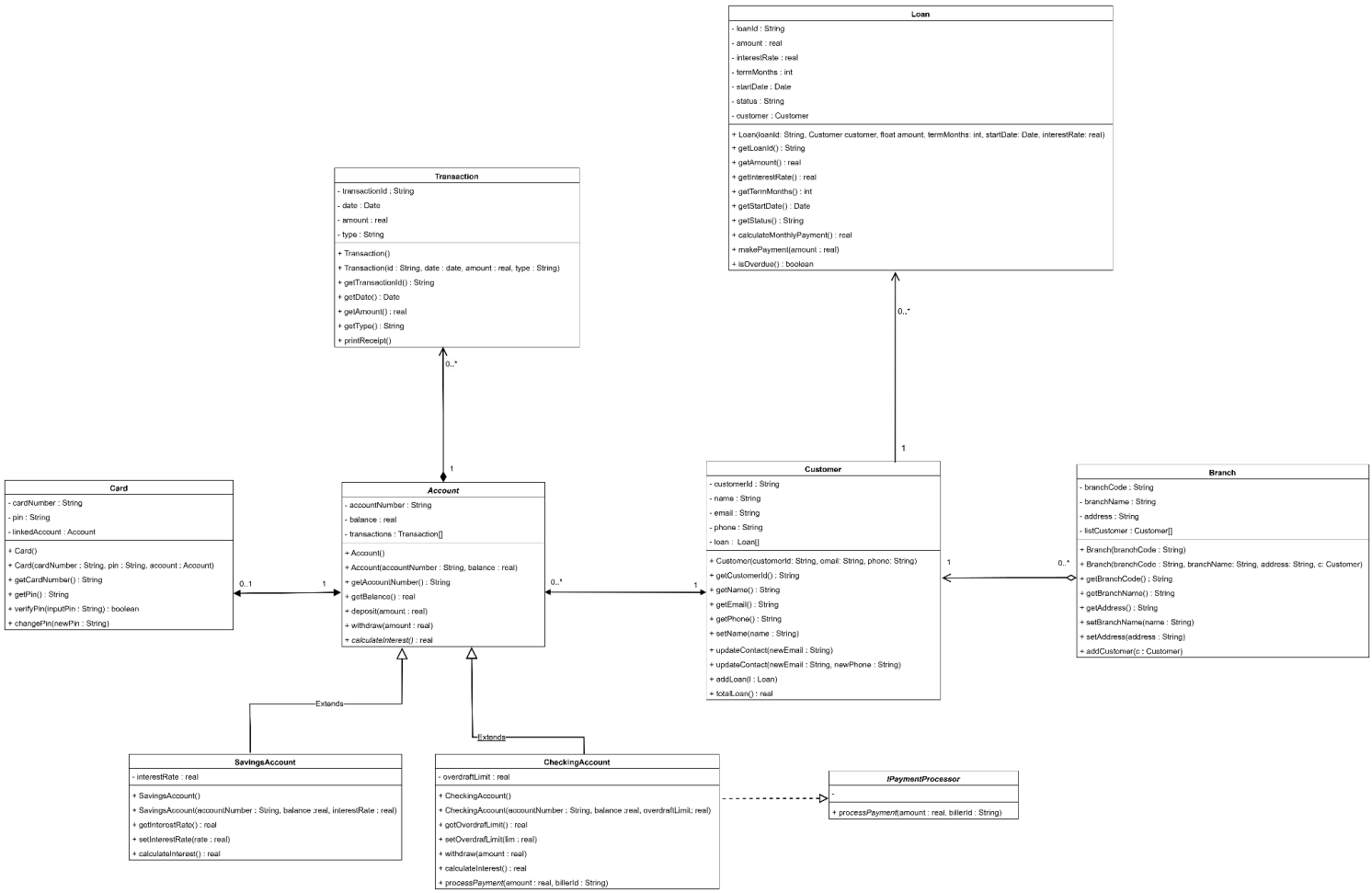
2026

A. Business Process

Sebuah bank memiliki kantor cabang (Branch) yang bertugas melayani nasabah (Customer). Setiap kantor cabang memiliki kode cabang, nama cabang, dan alamat. Setiap nasabah memiliki ID nasabah, nama, email, dan nomor telepon. Nasabah diperbolehkan memperbarui data kontaknya di sistem, baik hanya memperbarui email saja maupun memperbarui email beserta nomor telepon sekaligus. Nasabah dapat membuka rekening bank (Account) yang menyimpan data nomor rekening dan saldo. Rekening bank terdiri atas 2 jenis, yaitu rekening tabungan (Savings Account) dan rekening giro (Checking Account). Rekening tabungan memiliki persentase suku bunga, dan mendapatkan perhitungan bunga bulanan dari saldo dikalikan suku bunga. Rekening giro memiliki batas penarikan minus (overdraft limit) dan memiliki perhitungan bunga yang berbeda dengan tabungan.

Dalam operasionalnya, nasabah dapat menyetor maupun menarik dana dari rekening. Saat penyetoran, sistem harus memvalidasi bahwa nominal uang yang disetor tidak boleh bernilai negatif. Saat penarikan dana, rekening tabungan akan menolak transaksi dan mengeluarkan peringatan error jika nominal yang ditarik melebihi saldo yang tersedia. Sebaliknya, rekening giro mengizinkan penarikan dana melebihi saldo asalkan tidak melewati batas overdraft yang ditentukan. Setiap aktivitas penyetoran dan penarikan wajib dicatat sebagai riwayat transaksi (Transaction) yang memuat ID transaksi, tanggal, nominal, dan jenis transaksi. Riwayat transaksi ini merupakan bagian yang tidak terpisahkan dari rekening tersebut. Untuk kemudahan bertransaksi, sebuah rekening dapat dihubungkan dengan sebuah kartu ATM (Card) yang memiliki nomor kartu dan PIN. Selain itu, khusus untuk rekening giro, rekening tersebut terhubung dengan sebuah antarmuka pemrosesan pembayaran (Payment Processor) agar dapat digunakan untuk membayar tagihan ke pihak ketiga.

B. Class Diagram



C. Relasi antar Class

1. Account - Transaction

- Relasi : Komposisi

  Penjelasan : Transaction tidak bisa berdiri tanpa adanya Account, jika Account dihapus, maka Transaction yang ada di dalamnya juga ikut terhapus.

- Multiplicity : Account ke Transaction(0..*), Transaction ke Account(1)

  Penjelasan : Account bisa melakukan nol atau banyak Transaction, tetapi Setiap Transaction harus tepat terhubung ke satu Account.

- Navigability : Searah (Unidirectional)

  Penjelesan : Account mengetahui dan dapat mengakses informasi atau Method-method yang ada di class Transaction. Sebaliknya, Class Transaction tidak menyimpan referensi balik ke Account

2. Account - Card

- Relasi : Asosiasi

  Penjelasan : Account dan Card adalah Class Independen yang saling Berkolaborasi. Meskipun Account dihapus, secara fisik Card masih eksis walaupun isinya tidak bisa diakses lagi.

- Multiplicity : Account ke Card (0..1), Card ke Account (1)

  Penjelasan : Account bisa memiliki satu kartu atau tidak sama sekali, Tetapi setiap kartu harus tepat terhubung ke satu Account

- Navigability : Dua arah (Bidirectional)

  Penjelasan : Account perlu tahu Card mana yang terhubung untuk Keperluan aktivasi,blokir, dll. Sedangkan, Card perlu tahu Account ini milik siapa untuk keperluan mengenali sumber

Dana sehingga navigabilitynya adalah bidirectional.

3. Customer - Loan

- Relasi : Asosiasi
  Penjelasan : Customer berhubungan dengan Loan karena seorang Customer dapat memiliki data pinjaman. Relasi ini adalah asosiasi biasa karena tidak ada simbol diamond pada hubungan tersebut.

- Multiplicity : Customer ke Loan (0..*), Loan ke Customer (1)
  Penjelasan : Satu Customer dapat memiliki nol atau banyak Loan. Setiap Loan harus terkait dengan tepat satu Customer.

- Navigability : Searah (Unidirectional)
  Penjelasan : Customer mengetahui dan dapat mengakses Loan. Pada diagram ini, Loan tidak ditunjukkan menyimpan referensi balik ke Customer.

4. Customer - Account

- Relasi : Asosiasi
  Penjelasan : Customer berhubungan dengan Account karena Customer adalah pemilik atau pihak yang terkait dengan rekening. Hubungan ini termasuk asosiasi biasa, bukan komposisi atau agregasi.

- Multiplicity : Customer ke Account (0..*), Account ke Customer (1)
  Penjelasan : Satu Customer dapat memiliki nol atau banyak Account. Setiap Account harus terhubung ke tepat satu Customer.

- Navigability : Dua arah (Bidirectional)
  Penjelasan : Customer mengetahui dan dapat mengakses informasi pada Account. Account perlu tahu siapa pemiliknya untuk keperluan validasi, cetak nama,dsb.

5. Customer - Branch

- Relasi : Asosiasi
  Penjelasan : Customer terhubung dengan Branch karena setiap Customer terdaftar atau dilayani oleh suatu cabang bank. Ini merupakan asosiasi biasa.

- Multiplicity : Branch ke Customer (0..*), Customer ke Branch (1)
  Penjelasan : Satu Branch dapat melayani nol atau banyak Customer. Setiap Customer terhubung tepat ke satu Branch.

- Navigability : Searah (Unidirectional)
  Penjelasan : Customer mengetahui Branch tempat ia terdaftar atau dilayani. Pada diagram ini, Branch tidak ditunjukkan menavigasi balik langsung ke Customer melalui atribut referensi.

6. **SavingsAccount - Account**

- Relasi : Pewarisan
  Penjelasan : SavingsAccount adalah turunan dari Account. Artinya SavingsAccount mewarisi atribut dan method umum yang dimiliki Account, lalu menambahkan ciri khusus seperti interestRate dan calculateInterest().

- Multiplicity : Tidak berlaku
  Penjelasan : Relasi pewarisan tidak menggunakan multiplicity karena hubungan ini bukan hubungan jumlah objek, melainkan hubungan turunan kelas.

- Navigability : Tidak berlaku secara asosiasi
  Penjelasan : Panah segitiga kosong yang mengarah ke Account menunjukkan bahwa SavingsAccount mewarisi Account.

7. **CheckingAccount - Account**

- Relasi : Generalisasi atau Pewarisan
  Penjelasan : CheckingAccount adalah turunan dari Account. Class ini mewarisi atribut dan method umum dari Account, lalu menambahkan fitur khusus seperti overdraftLimit dan processPayment().

- Multiplicity : Tidak berlaku
  Penjelasan : Karena ini relasi pewarisan, multiplicity tidak digunakan.

- Navigability : Tidak berlaku secara asosiasi
  Penjelasan : Panah segitiga kosong ke arah Account menunjukkan bahwa CheckingAccount adalah subclass dari Account.

8. **CheckingAccount - IPaymentProcessor**

- Relasi : Implementasi Interface
  Penjelasan : CheckingAccount mengimplementasikan interface IPaymentProcessor. Ini berarti CheckingAccount wajib menyediakan method **`processPayment(amount, billerId)`** sesuai kontrak yang didefinisikan oleh interface.

- Multiplicity : Tidak berlaku
  Penjelasan : Relasi interface tidak menggunakan multiplicity karena bukan hubungan jumlah objek.
- Navigability : Tidak berlaku secara asosiasi
  Penjelasan : Garis putus putus dengan segitiga kosong ke arah IPaymentProcessor menunjukkan bahwa CheckingAccount mengimplementasikan interface tersebut.

D. Penggunaan Exception Handling

Pada studi kasus Bank, penggunaan Exception Handling diimplementasikan pada method deposit dan withdraw pada class Account. Pada method deposit, exception dilempar apabila nominal yang dimasukkan bernilai nol atau negatif, sedangkan pada method withdraw terdapat dua exception, yaitu akan melempar exception jika nominal yang dimasukkan bernilai nol atau negatif dan saldo tidak mencukupi untuk melakukan penarikan.

E. Overloading dan Overriding

- Overloading
  Overloading terdapat dalam fitur Pembaruan Data Customer
  Penjelasan: Seiring berjalannya waktu, nasabah mungkin mengganti informasi kontaknya. Sistem memfasilitasi nasabah untuk memperbarui datanya secara fleksibel. Nasabah bisa memilih untuk hanya memperbarui alamat email saja, atau memperbarui email sekaligus nomor telepon genggamnya dalam satu waktu pengerjaan (Method Overloading).

- Overriding
  Overriding terdapat dalam fitur Penarikan Dana (Withdraw) pada CheckingAccount dan SavingsAccount
  Penjelasan: Sistem menyediakan fungsi dasar penarikan dana pada rekening induk yang sekadar mencegah input nominal negatif. Namun, setiap jenis rekening memiliki aturan validasi yang spesifik. Rekening tabungan harus mengecek kecukupan saldo, sementara rekening giro mengecek batas toleransi hutang (*overdraft*). Oleh karena itu, kedua jenis rekening anak tersebut menimpa

fungsi penarikan dasar dari kelas induknya untuk menerapkan aturan transaksinya masing-masing (Method Overriding).

F. Screenshoot Source Code

1. Account

```java
import java.time.Instant;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public abstract class Account {
   /* Atribut */

   private String accountNumber;
   private double balance;
   private List<Transaction> transactions = new ArrayList<>();

   /*Method */

   /*Konstruktor */
    public Account(){


    }
    public Account(String accountNumber, double balance){
        this.accountNumber = accountNumber;
        this.balance = balance;

    }

    /*Selektor */
    public String getAccountNumber() {
        return this.accountNumber;
    }
    public double getBalance() {
        return this.balance;
    }



    public void deposit(double amount) throws Exception {
        if(amount <=0) {
            throw new Exception("Nominal deposit harus lebih dari nol");
        }
        balance += amount;

        transactions.add(new Transaction("T" + Instant.now().getEpochSecond(), LocalDate.now(),amount,"Deposit"));
    }

    public void withdraw(double amount) throws Exception {
        if (amount <= 0) {
            throw new Exception("Nominal penarikan harus lebih dari nol");
        }

        this.balance -= amount;
        transactions.add(new Transaction("T" + Instant.now().getEpochSecond(), LocalDate.now(),amount,"Withdraw"));
    }
    public abstract double calculateInterest();


}

```

2. Transaction

src > Transaction.java > Transaction > getAmount()

```java
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class Transaction {

    /* Atribut */

   private String transactionId;
   private LocalDate date;
   private double amount;
   private String type;

   /*Method */

   /*Konstruktor */
    public Transaction(){


    }
    public Transaction(String id, LocalDate date, double amount, String type) {
        this.transactionId = id;
        this.date = date;
        this.amount = amount;
        this.type = type;
    }

    /*Selektor */
    public String getTransactionId() {
        return this.transactionId;
    }
    public LocalDate getDate() {
        return this.date;
    }
    public double getAmount() {
        return this.amount;
    }
    public String getType() {
        return this.type;
    }



    public void printReceipt() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern(pattern: "dd MMMM yyyy");
        String formattedDate = date.format(formatter);
        System.out.println(x: "==================================");
        System.out.println(x: "         STRUK TRANSAKSI          ");
        System.out.println(x: "==================================");
        System.out.println("ID Transaksi : " + this.transactionId);
        System.out.println("Tanggal      : " + formattedDate);
        System.out.println("Tipe         : " + this.type);
        System.out.println("Nominal      : Rp" + String.format(format: "%,.2f", this.amount));
        System.out.println(x: "==================================");
        System.out.println(x: "   Terima Kasih Telah Bertransaksi ");
        System.out.println(x: "==================================\n");
    }
}
```

3. Card

src > Card.java > Card

```java
public class Card {

    /* Atribut */

   private String cardNumber;
   private String pin;
   private Account linkedAccount;

   /*Method */

   /*Konstruktor */
    public Card(){


    }
public Card(String cardNumber, String pin, Account account) {
        this.cardNumber = cardNumber;
        this.pin = pin;
        this.linkedAccount = account;

    }

    /*Selektor */
   public String getCardNumber() {
        return this.cardNumber;
    }
    public String getPin() {
        return this.pin;
    }

    public Account getLinkedAccount() {
        return this.linkedAccount;
    }




    public boolean verifyPin(String inputPin) {
        return this.pin == inputPin;
    }

    public void changePin(String newPin) {
        if (newPin != null && newPin.length() == 6 && newPin.matches(regex: "\\d+")) {
            this.pin = newPin;
            System.out.println(x: "PIN berhasil diubah.");
        } else {
            System.out.println(x: "Gagal: PIN harus terdiri dari 6 digit angka.");
        }
    }

}
```

4. Customer

src > Customer.java > Customer

```java
public class Customer {
  private String customerId;
  private String name;
  private String email;
  private String phone;
  private java.util.List<Loan> loan;

  /* Konstruktor */
  public Customer(String customerId, String email, String phone) {
    this.customerId = customerId;
    this.email = email;
    this.phone = phone;
    this.loan = new java.util.ArrayList<>();
  }

  /* Selektor */
  public String getCustomerId() {
    return customerId;
  }

  public String getName() {
    return name;
  }

  public String getEmail() {
    return email;
  }

  public String getPhone() {
    return phone;
  }

  public java.util.List<Loan> getLoan() {
    return loan;
  }

  /* Mutator */
  public void setName(String name) {
    this.name = name;
  }

  /* Method Tambahan */
  public void updateContact(String newEmail) {
    this.email = newEmail;
  }

  //Overloading
  public void updateContact(String newEmail, String newPhone) {
    this.email = newEmail;
    this.phone = newPhone;
  }

  public void addLoan(Loan l) {
    loan.add(l);
  }

  public double totalLoan() {
    double totAmount = 0;
    for(Loan l : loan) {
      totAmount += l.getAmount();
    }
    return totAmount;
  }
}
```

5. Loan

```java
public class Loan {
  private String loanId;
  private double amount;
  private double interestRate;
  private int termMonths;
  private java.util.Date startDate;
  private String status;
  private Customer customer;

  /* Konstruktor */
  public Loan(String loanId, double amount, Customer customer, int termMonths, java.util.Date startDate, double interestRate) {
    this.loanId = loanId;
    this.amount = amount;
    this.customer = customer;
    this.termMonths = termMonths;
    this.startDate = startDate;
    this.interestRate = interestRate;
    this.status = "Aktif";
  }

  /* Selektor */
  public String getLoanId() {
    return loanId;
  }

  public double getAmount() {
    return amount;
  }

  public double getInterestRate() {
    return interestRate;
  }

  public int getTermMonths() {
    return termMonths;
  }

  public java.util.Date getStartDate() {
    return startDate;
  }

  public String getStatus() {
    return status;
  }

  public Customer getCustomer() {
    return customer;
  }

  /* Method Tambahan */
  public double calculateMonthlyPayment() {
    if(amount <= 0 || termMonths <= 0) {
      return 0.0f;
    }

    double cicilan = interestRate / 12 / 100;

    if(interestRate == 0){
      return amount / termMonths;
    }

    return (double) (amount * cicilan / (1-Math.pow(1 - cicilan, -termMonths)));
  }

  public void makePayment(double paymentAmount) {
    if(paymentAmount > 0) {
      amount -= paymentAmount;

      if(amount <= 0) {
        amount = 0;
        status = "Lunas";
      }
      else{
        status = "Aktif";
      }

    }
  }

  public boolean isOverdue() {
    if(startDate == null){
      System.out.println("Start date belum diatur.");
      return false;
    }

    java.util.Calendar cal = java.util.Calendar.getInstance();
    cal.setTime(startDate);
    cal.add(java.util.Calendar.MONTH, termMonths);

    java.util.Date dueDate = cal.getTime();
    java.util.Date today = new java.util.Date();

    return today.after(dueDate) && status == "Aktif";
  }

}
```

6. Branch

```java
import java.util.ArrayList;

public class Branch {
        /* Atribut */
        private String branchCode;
        private String branchName;
        private String address;
        ArrayList<Customer> listCustomer;

        /* Konstruktor */
        public Branch(String branchCode){
            this.branchCode = branchCode;
            this.branchName = "";
            this.address = "";
            this.listCustomer = new ArrayList<>();
        }

        public Branch(String branchCode, String branchName, String address, Customer c){
            this.branchCode = branchCode;
            this.branchName = branchName;
            this.address = address;
            this.listCustomer = new ArrayList<>();
            this.listCustomer.add(c);
        }

        /* Selektor */
        public String getBranchCode(){
            return this.branchCode;
        }

        public String getBranchName(){
            return this.branchName;
        }

        public String getAddress(){
            return this.address;
        }

        /* Mutator */
        public void setBranchName(String branchName){
            this.branchName = branchName;
        }

        public void setAddress(String address){
            this.address = address;
        }

        /* Fungsi */
        public void addCustomer(Customer c){
            this.listCustomer.add(c);
        }
}
```

7. IpaymentProcessor

```java
public interface IPaymentProcessor {
    /* Method */
    boolean processPayment(double amount, String billerId);
}
```

8. CheckingAccount

```java
public class CheckingAccount extends Account implements IPaymentProcessor {
    /* Atribut */
    private double overdraftLimit;

    /* Method */
    public CheckingAccount(){
        super();
        overdraftLimit = 500000;
    }

    public CheckingAccount(String accountNumber, double balance, double overdraftLimit) {
        super(accountNumber, balance);
        this.overdraftLimit = overdraftLimit;
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    public void setOverdraftLimit(double overdraftLimit) {
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public void withdraw(double amount) throws Exception {
        if (amount > overdraftLimit + getBalance()) {
            throw new Exception("Penarikan gagal: Saldo dan Overdraft tidak cukup.");
        }
        super.withdraw(amount);
    }

    @Override
    public boolean processPayment(double amount, String billerId) {
        try {
            this.withdraw(amount);
            System.out.println("Pembayaran sebesar " + amount + " ke " + billerId + " berhasil.");
            return true;
        } catch (Exception e) {
            System.out.println("Pembayaran gagal: Saldo dan Overdraft tidak cukup.");
            return false;
        }
    }

    @Override
    public double calculateInterest() {
        return 0;
    };

}
```

9. SavingAccount

```java
public class SavingsAccount extends Account {
    /* Atribut */
    private double interestRate;

    /* Method */
    public SavingsAccount() {
        super();
        interestRate = 0.05;
    }

    public SavingsAccount(String accountNumber, double balance, double interestRate) {
        super(accountNumber, balance);
        this.interestRate = interestRate;
    }

    public double getInterestRate() {
        return this.interestRate;
    }

    public void setInterestRate(double rate) {
        this.interestRate = rate;
    }

    @Override
    public double calculateInterest() {
        return super.getBalance() * this.interestRate;
    }

    @Override
    public void withdraw(double amount) throws Exception {
        if (amount > getBalance()) {
            throw new Exception("Penarikan gagal: Saldo tidak cukup.");
        }
        super.withdraw(amount);
    }
}
```

G. Screenshoot hasil running main class

```
$ java -cp bin Main
=======CUSTOMER======
Nama Customer : Anton Binsar Panjaitan
Nomor Telepon  : 086745678453
Email  : antonpanjaitan@gmail.com

Nama Customer : Sherina Izzati
Nomor Telepon  : 086745678453
Email  : sherina@students.ub.ac.id

=======ACCOUNT========

=======SAVINGACCOUNT========
Nomor Rekening : SA-001
Saldo : Rp5,000,000.00

Deposit Berhasil
==================================
         STRUK TRANSAKSI
==================================
ID Transaksi    : T1774971382
Nomor Rekening  : SA-001
Tanggal         : 31 March 2026
Tipe            : Deposit
Nominal         : Rp2,000,000.00
==================================
   Terima Kasih Telah Bertransaksi
==================================

Total Saldo : Rp7,000,000.00

==========GIROACCOUNT==========
Nomor Rekening : CA-001
Saldo : Rp2,000,000.00
Overdraft Limit : Rp1,000,000.00

Withdraw berhasil
==================================
         STRUK TRANSAKSI
==================================
ID Transaksi    : T1774971382
Nomor Rekening  : CA-001
Tanggal         : 31 March 2026
Tipe            : Withdraw
Nominal         : Rp1,500,000.00
==================================
   Terima Kasih Telah Bertransaksi
==================================
```

```
Saldo : Rp500,000.00

=============INTERFACE=============
Pembayaran sebesar Rp250,000.00 ke UNDIP-1957 berhasil.
Saldo : Rp250,000.00

=============CARD=============
Nomor kartu : 4111-1111-1111-1111
Akun terhubung : SA-001
Verifikasi PIN: Benar
PIN berhasil diubah.

=============LOAN=============
Pinjaman ID                   : L001
Jumlah pinjaman               : Rp10,000,000.00
Cicilan/bulan                 : Rp852,603.31
Status                        : Aktif
Terlambat                     : false
Total pinjaman Customer C001  : Rp10,000,000.00

Status setelah bayar : Aktif
Total sisa cicilan   : Rp500,000.00

========= CABANG ===========
Kode cabang : BUB-01
Nama cabang : Bank Brawijaya Kota Malang
Alamat      : Jl. Veteran No.1 Kota Malang
Daftar Customer di Cabang Bank Brawijaya Kota Malang:
- Anton Binsar Panjaitan
- Sherina Izzati
```